

Adrena

Smart Contract Security Assessment

December 2024

Prepared for:

Adrena Foundation

Prepared by:

Offside Labs

Ronny Xing

Yao Li

Sirius Xie

Siji Feng





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	7
4.1	Insufficient Position Price Precision	7
4.2	Governance Shadow Tokens Can Be Minted Infinitely	8
4.3	Miscalculation for Position Price in Increase Position	9
4.4	Incorrect Custody Used for Interest Calculation	10
4.5	Insufficient Thread Account Verification	11
4.6	Cumulative Interests of Short Positions are Not Included in AUM	12
4.7	Incorrect Update of PositionsAccounting.cumulative_interest_usd	13
4.8	Position Increase Instructions Can Reset Liquidation Fee to Zero	14
4.9	Incorrect Update of Collateral Custody Borrow Rate	15
4.10	Missing SL Slippage Check in close_position_short Instruction	16
4.11	Add Liquidity LP Fees Distribution is Inaccurate	16
4.12	AUM Profit Deduction Calculation Error	17
4.13	LM LockedStake With Zero Locked Days Can Not Claim Rewards	18
4.14	Excessive LP Fee Distribution to Locked LP	19
4.15	Increase Position with Swap Checks Incorrect Custody Flag	20
4.16	Should Use High Price to Calculate Collateral Withdrawal	21
4.17	Short Positions Should Also Use High Collateral Price for Accounting Update	21
4.18	Set SL Instruction Should Ensure close_position_price Parameter Is Not Zero	22
4.19	Remove Liquidity LP Fees Distribution is Inaccurate	23
4.20	Borrow Rate Not Updated After Fee Distribution	23
4.21	External Fees Not Considered in AUM Check	25
4.22	Possible Updating on Unintended LockedStakes when Claiming Genesis Rewards	26
4.23	Liquidity Operations Moving Towards Target Ratio May be Rejected	27
4.24	Unable to Guarantee Operations Sequence About Staking	28
4.25	Informational and Undetermined Issues	29
5	Disclaimer	35



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of Adrena smart contracts, starting on October 21, 2024 and concluding on November 29, 2024.

Project Overview

Adrena is a peer-to-pool perpetuals DEX with 100x leverage and 100% revenue share to token holders. Adrena is a decentralized spot and perpetual exchange built on Solana with low swap fees and ZERO price impact trades.

Trading is supported by a multi-asset pool that earns liquidity providers fees from lending to traders, swap fees, and additional token incentives.

The application is supported by a 2-token model; \$ADX (governance token) and \$ALP (LP token). \$ADX gives users voting rights to control the platform through the Adrena DAO. \$ADX also participates in 20% revenue share of the platform. \$ALP is the representation of the multi-asset pool and earns 70% of the platform revenue.

Audit Scope

The assessment scope contains mainly the smart contracts of the adrena program for the Adrena project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Adrena
 - Codebase: <https://github.com/AdrenaFoundation/adrena>
 - Commit Hash: 34d55495d68eb538a7b1d62ef180916be54d76ec

We listed the files we have audited below:

- Adrena
 - programs/adrena/src/**/*rs

Findings

The security audit revealed:

- 3 critical issues
- 3 high issues
- 8 medium issues
- 10 low issues
- 12 informational issues



Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Insufficient Position Price Precision	Critical	Fixed
02	Governance Shadow Tokens Can Be Minted Infinitely	Critical	Fixed
03	Miscalculation for Position Price in Increase Position	Critical	Fixed
04	Incorrect Custody Used for Interest Calculation	High	Fixed
05	Insufficient Thread Account Verification	High	Fixed
06	Cumulative Interests of Short Positions are Not Included in AUM	High	Fixed
07	Incorrect Update of PositionsAccounting.cumulative_interest_usd	Medium	Fixed
08	Position Increase Instructions Can Reset Liquidation Fee to Zero	Medium	Fixed
09	Incorrect Update of Collateral Custody Borrow Rate	Medium	Fixed
10	Missing SL Slippage Check in close_position_short Instruction	Medium	Fixed
11	Add Liquidity LP Fees Distribution is Inaccurate	Medium	Fixed
12	AUM Profit Deduction Calculation Error	Medium	Fixed
13	LM LockedStake With Zero Locked Days Can Not Claim Rewards	Medium	Fixed
14	Excessive LP Fee Distribution to Locked LP	Medium	Fixed
15	Increase Position with Swap Checks Incorrect Custody Flag	Low	Fixed
16	Should Use High Price to Calculate Collateral Withdrawal	Low	Fixed
17	Short Positions Should Also Use High Collateral Price for Accounting Update	Low	Fixed
18	Set SL Instruction Should Ensure close_position_price Parameter Is Not Zero	Low	Fixed
19	Remove Liquidity LP Fees Distribution is Inaccurate	Low	Fixed
20	Borrow Rate Not Updated After Fee Distribution	Low	Fixed
21	External Fees Not Considered in AUM Check	Low	Fixed
22	Possible Updating on Unintended LockedStakes when Claiming Genesis Rewards	Low	Fixed



ID	Title	Severity	Status
23	Liquidity Operations Moving Towards Target Ratio May be Rejected	Low	Fixed
24	Unable to Guarantee Operations Sequence About Staking	Low	Acknowledged
25	No Checks on Opening Long Positions With Stable Custodies	Informational	Fixed
26	Suggest Checking Both Positions Open Time and Update Time	Informational	Fixed
27	close_position_short Not Update collateral_custody.collected_fees.borrow_usd	Informational	Fixed
28	OpenOrIncreasePositionWithSwapShort Instruction Missing protocol_fee_recipient Check	Informational	Fixed
29	Suggested Add Price Validation for SL Parameters	Informational	Fixed
30	Incorrect Validation in remove_pool	Informational	Fixed
31	Incorrect Validation in remove_custody	Informational	Fixed
32	Incorrect Unique Custody Check in Pool::validate	Informational	Fixed
33	Recommend to Check if the Locked State Has Ended When Finalizing With early_exit	Informational	Fixed
34	OraclePrice::normalize Doesn't Update the Confidence	Informational	Acknowledged
35	Instruction add_vest Can Add Vests Which Can be Almost Fully Unlocked Immediately	Informational	Acknowledged
36	Insufficient Precision of PositionsAccounting.total_quantity	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Insufficient Position Price Precision

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Precision

Description

There is the `position.price` update implementation in `increase_position_long/short` instructions:

```
245     position.price = math::checked_as_u64::<u128>(
246         ((position.price as u128 * position.size_usd as u128)
247          + (token_trade_price.price as u128 * size_usd as u128))
248         / (position.size_usd as u128 + size_usd as u128),
249     )?;
```

[programs/adrena/src/instructions/public/position/increase_position_long.rs#L245-L249](#)

This implementation always uses floor rounding when calculating the new position price, which is converted to a u64 price with an exponent of -10.

This means changes to the position's opening price smaller than $1e-10$ resulting from increasing positions will be truncated and ignored, while the `position.size_usd` will still increase normally.

Impact

For long positions, using floor rounding for price updates is harmful to the protocol.

Under normal circumstances, a price error of $1e-10$ would have no impact. However, since some meme coins have extremely low prices, these errors can accumulate through multiple position increases. The accumulated price errors can exceed 10% of the meme coin's price. Attackers can exploit this by withdrawing these accumulated errors all at once during position closure, thereby stealing tokens from the custody pool.

Proof of Concept

The program has already listed Bonk as a custody on mainnet. In the following PoC, we also use Bonk as the custody for position opening, and the PoC uses two real Bonk prices 189512 and 210000.

Results:

- 0 position increases: profit 23690328215098, as the base;



- 100 increases: profit 23796437014817, profit increased by 0.448%;
- 200 increases: profit 23902545819289, profit increased by 0.896%

Recommendation

The price updates in `increase_position_long` can be modified to use ceiling rounding. However, note that this issue fundamentally stems from the insufficient precision of exponent -10 when handling certain meme coin prices, though this may already be the maximum precision possible within u64's bit width.

Mitigation Review Log

Adrena Team: [PR-166](#)

We decided to ceil on increase position long, the effect is that the loss due to imprecision is transferred to the user.

Offside Labs: Fixed.

4.2 Governance Shadow Tokens Can Be Minted Infinitely

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic

Description

In the SPL governance program, the relationship between the governance realm and the community mint is not one-to-one.

Multiple Governance Realms can be created for one community mint. The adrena program performs no additional validation for the account `governance_realm` when integrating the SPL governance `DepositGoverningTokens` instruction. This allows attackers to input forged realms to mint gov tokens into realms not controlled by adrena.

Impact

This enables attackers to mint unlimited governance shadow tokens, ultimately gaining control of the DAO.

This attack vector relies on the following two prerequisites:

1. Attackers can set `token_type` to `GoverningTokenType::Liquid` in their forged governance realm, allowing them to withdraw gov tokens from the gov holding token account.



2. `Cortex::remove_governing_power` method sets a maximum limit on gov token burning to handle user external operation scenarios.

```
658 let revoke_amount = min(  
659     amount,  
660     token_owner_record_data.governing_token_deposit_amount,  
661 );  
  
programs/adrena/src/state/cortex.rs#L658-L661
```

Recommendation

Add constraints for the `governance_realm` address.

Mitigation Review Log

Adrena Team: [PR-186](#)

We will release this fix along with release/18 things.

Offside Labs: removing the code that takes the min on the `revoke_amount` will cause the scenario where the user self revoke some token externally always fails when these users want to unstake LM.

4.3 Miscalculation for Position Price in Increase Position

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic

Description

In the `increase_position_long` and `increase_position_short` instructions, the following formula is used to recalculate the average entry price when increasing a position:

```
239 // calculate the new position price (weighted average of the old  
    // and new prices)  
240 position.price = math::checked_as_u64::<u128>(  
241     (math::checked_ceil_div(  
242         (position.price as u128 * position.size_usd as u128)  
243         + (token_trade_price.price as u128 * size_usd as u128),  
244         position.size_usd as u128 + size_usd as u128,  
245     ))?,  
246 );  
  
programs/adrena/src/instructions/public/position/increase_position_  
long.rs#L239-L246
```



However, the formula does not account for the PnL of origin position. This oversight causes the recalculated entry price to be inaccurate, leading to miscalculated PnL for users or the pool.

Impact

All increased positions will have incorrect entry price, and users or pool will bear the losses due to the miscalculated PnL.

Recommendation

For long position, the weight of origin position should be added by PnL profit. For short position, the PnL profit should be subtracted from the weight.

Mitigation Review Log

Adrena Team: Fixed in [PR-190](#).

Offside Labs: Fixed.

4.4 Incorrect Custody Used for Interest Calculation

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic

Description

`update_accounting_after_remove_position_short` function uses `self` custody to calculate accumulated unrealized interest and interest snapshot, where `self` custody refers to trading custody instead of collateral custody.

```
577     pub fn update_accounting_after_remove_position_short(  
578         ...  
579     ) -> Result<()> {  
580         let cumulative_interest_snapshot =  
581             self.get_cumulative_interest(current_time)?;  
582         let position_unrealized_interest_usd =  
583             self.get_interest_amount_usd(position, current_time)?;
```

[programs/adrena/src/state/custody.rs#L577-L585](#)

Impact

This causes `collateral_custody.short_positions` to update `cumulative_interest_usd` and `cumulative_interest_snapshot` with incorrect values, ultimately resulting in



incorrect AUM calculations that may cause losses to LPs.

[programs/adrena/src/state/custody.rs#L616-L622](#)

Recommendation

Use `collateral_custody` to calculate variables related to interest.

Mitigation Review Log

Adrena Team: [PR-178](#) fixing the issue.

Offside Labs: Fixed

4.5 Insufficient Thread Account Verification

Severity: High

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

Users can use Sablier threads to automate TP/SL and staking operations. In Sablier instructions, except for the `thread.authority` signature verification, all other checks, such as PDA address validation, are introspective. This means that `thread.authority` can directly manipulate all threads under its ownership. In Adrena, all threads share one authority, which is the PDA corresponding to the seed “transfer_authority”.

The issue is that, Thread accounts created in the `staking_p` and `position` components can be interchanged. Furthermore, since thread IDs between these two components cannot guarantee uniqueness (due to different domains), validations based on thread IDs become ineffective.

Threads created by admin instructions may also be affected.

Impact

Instructions that can manipulate any thread due to lack of thread account validation:

- `add_liquid_stake`
- `add_locked_stake`
- `finalize_locked_stake`
- `remove_liquid_stake`



- `remove_locked_stake`: this instruction checks `thread.id == user_staking.load()?.stakes_claim_cron_thread_id` . However, since `stakes_claim_cron_thread_id` comes from user input, it can be used to manipulate threads created by the position component, despite having different domains.
- `upgrade_locked_stake`

These instructions can be used to update, pause, and delete arbitrary threads, and steal the rent and reserved CU costs.

After a perpetual trading user sets TP/SL orders for their position, these instructions can be used to delete their TP/SL threads, resulting in:

1. Users' set TP/SL orders cannot be automatically executed, resulting in potential losses.
2. Due to `utils::validate_sablier_thread_account` check, users cannot manually close positions unless exploiting this bug.

Recommendation

Ensure thread IDs in the `staking_p` and `position` components are unique. It's also recommended to add a domain when creating threads in `staking_p` component to prevent potential conflicts.

Thoroughly validate that the thread ID matches what is stored in the user-related Account.

Mitigation Review Log

Adrena Team: Fixed by removing Sablier integration in release/15.

Offside Labs: Fixed.

4.6 Cumulative Interests of Short Positions are Not Included in AUM

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic

Description

In the AUM calculation `get_assets_under_management_usd` , interest calculation for short positions is in a separate branch:

[programs/adrena/src/state/pool.rs#L912-L916](#)



```
let collective_position = custody.get_collective_position(Side::Short)?;  
let interest_usd =  
    custody.get_interest_amount_usd(&collective_position, current_time)?;  
  
pool_amount_usd += interest_usd as u128;
```

`custody.short_positions.cumulative_interest_usd` is missing here. The `custody.get_interest_amount_usd` only returns the interest increase since the last `cumulative_interest_snapshot` update

Impact

This would allow opening/increasing short positions to immediately deduct all accumulated interest from AUM, potentially enabling attackers to manipulate LP prices for arbitrage.

Proof of Concept

The PoC test below shows that AUM significantly decreases after opening the second short position.

Recommendation

For stable coin custody, interest should also include

```
custody.short_positions.cumulative_interest_usd .
```

Mitigation Review Log

Adrena Team: [PR-178](#) fixing the issue.

Offside Labs: Fixed.

4.7 Incorrect Update of `PositionsAccounting.cumulative_interest_usd`

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic

Description

In functions `update_accounting_after_open_position_long/short`, when updating `accounting.cumulative_interest_usd`, only the result of `Custody::get_interest_amount_usd(&collective_position, current_time)` is added. It doesn't take into consideration of `position.unrealized_interest_usd` being non-zero if its called by the `increase_position_long/short` instructions.



And only the unrealized interest of positions have been deducted from the position accounting of the collateral custody when calling `update_accounting_after_remove_position_long/short` functions. It also fails to handle cases where `position.unrealized_interest_usd` is non-zero.

Impact

As a result, when increasing a position, the accumulated borrowing interest of the position is directly deducted from AUM, and interest accumulation restarts from 0 based on current time and position size.

And when closing a position, the `unrealized_interest_usd` of the position will remain permanently in `accounting.cumulative_interest_usd`, even though this portion of the borrow fee has already been distributed.

This will cause a momentary decrease/increase in AUM and LP value, potentially resulting in losses for LPs.

Recommendation

Deduct `position.unrealized_interest_usd` in `update_accounting_after_remove_position_long/short` and add it back in `update_accounting_after_open_position_long/short`.

Mitigation Review Log

Adrena Team: Related to 02, we will do a full assessment, [PR-178](#) fixing the issue.

Offside Labs: The `update_accounting_after_remove_position_long` function doesn't subtract `position.unrealized_interest_usd`
[programs/adrena/src/state/custody.rs#L703-L703](#)

Adrena Team: Fixed in [PR-186](#).

Offside Labs: Fixed.

4.8 Position Increase Instructions Can Reset Liquidation Fee to Zero

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic

Description

When `increase_position_long` and `increase_position_short` instructions update position state, `position.liquidation_fee_usd` is directly overwritten instead of being increased:



283 `position.liquidation_fee_usd = liquidation_fee_usd;`

`programs/adrena/src/instructions/public/position/increase_position_`
`short.rs#L283-L283`

Impact

If the input `params.collateral` is zero, the `liquidation_fee_usd` will be zero.

Users can use this approach to avoid liquidation fees during liquidation, resulting in reduced protocol fee collection.

Recommendation

Use `+=` instead of `=`.

Mitigation Review Log

Adrena Team: [PR-169](#), Indeed, updated to `+=`.

Offside Labs: Fixed.

4.9 Incorrect Update of Collateral Custody Borrow Rate

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic

Description

The `increase_position_short` instruction updates custody's `borrow_rate_state` instead of `collateral_custody`'s at the end.

332 `custody.update_borrow_rate(current_time)?;`

`programs/adrena/src/instructions/public/position/increase_position_`
`short.rs#L332-L332`

Impact

Due to increased `locked_amount` from position increase, `borrow_rate_state.current_rate` will grow. However, `collateral_custody`'s `borrow_rate_state` is not properly updated, resulting in subsequent borrowing fees being lower than expected.

Recommendation

It should be `collateral_custody.update_borrow_rate(current_time)?;`.



Mitigation Review Log

Adrena Team: [PR-178](#) fixing the issue.

Offside Labs: Fixed.

4.10 Missing SL Slippage Check in `close_position_short` Instruction

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The `close_position_short` instruction does not call `position.stop_loss_slippage_ok` function, causing the `stop_loss_close_position_price` set by owner to become ineffective.

Impact

When other users (not a Sablier thread) execute SL orders, the execution will not be protected by `position.stop_loss_close_position_price`. This violates the expectations of the position owner.

Recommendation

Add `position.stop_loss_slippage_ok(token_trade_price.price)` check.

Mitigation Review Log

Adrena Team: [PR-170](#) Fixed as suggested.

Note: Luckily should not have cause issues as the slippage is currently not part of the UI for users, and we use a simple 1%, and the execution has been pretty tight.

Offside Labs: Fixed.

4.11 Add Liquidity LP Fees Distribution is Inaccurate

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic



Description

Instruction `add_liquidity` calls `calculate_fee_distribution` function after minting LP tokens. This leads to inaccurate fee distribution due to changes in AUM.

For the `add_liquidity` instruction, the minting LP amount is $\text{no_fee_amount USD} / \text{AUM} * \text{lp_token_mint.supply}$. However, after fee distribution is completed, new AUM will become $\text{AUM} + \text{no_fee_amount USD} + \text{fee_distribution.lp_organic_fee USD}$. This means newly minted LP tokens also claim a portion of the `lp_organic_fee` share.

Impact

This issue in the `add_liquidity` instruction causes regular LPs to receive less fees than expected.

Recommendation

Recommend calculating fee distribution before calculating LP amount to mint, and use $\text{pool_amount_usd} + \text{lp_organic_fee USD}$ as denominator for calculating LP amount to mint.

Mitigation Review Log

Adrena Team: Fixed in [PR-192](#).

Offside Labs: Fixed.

4.12 AUM Profit Deduction Calculation Error

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic

Description

The `Pool.get_assets_under_management_usd` method iterates through all custodies of the pool, and at the end of the loop directly subtracts the profits from global short/long positions' PnL corresponding to that custody from `pool_amount_usd`.

```
982 pool_amount_usd = pool_amount_usd.saturating_sub(long_pnl.profit_usd as
    ↪ u128);
983 pool_amount_usd = pool_amount_usd.saturating_sub(short_pnl.profit_usd as
    ↪ u128);
```

[programs/adrena/src/state/pool.rs#L982-L983](#)



Although we can assume profits are always capped by `custody.asset.locked`, it overlooks that the locked amount of short positions exists in the stablecoin collateral custody instead of the trading custody.

Therefore, the AUM calculation here is no longer protected by `custody.asset.locked`. It could be reduced to 0 by `saturating_sub`, thus ignoring some AUM losses caused by traders profits.

Impact

When stablecoin custodies in the `pool.custodies` array are not positioned before other custodies, AUM calculation may ignore losses from short positions profits, leading to significant overestimation. This could result in losses for LPs.

Recommendation

Recommend storing profits and losses from PnL in separate local variables. After the loop, AUM is equal to `AUM + pnl.loss_usd - pnl.profit_usd`. It's recommended to use `saturating_sub` here.

Mitigation Review Log

Fixed in [PR-206](#) (merged in release/20)

4.13 LM LockedStake With Zero Locked Days Can Not Claim Rewards

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic

Description

If users add LM locked state with params `AddLockedStakeParams.locked_days=0`, it will th following locked staking option parameters:

```
locked_days: 0,
reward_multiplier: 1
lm_reward_multiplier: 0
vote_multiplier: 1
```

So, the `end_time` of the `new_locked_stake` added will be `current_time`.

The following function `qualifies_for_rewards_from` is used to check if the locked state can claim the rewards in the certain round:

```
131     && staking_round.end_time <= self.end_time
132     && staking_round.start_time < self.end_time
```



[programs/adrena/src/state/user_staking.rs#L131-L132](#)

This condition for locked states with zero locked days will always be false.

Impact

Users lock LM tokens with zero locked days can't claim any rewards from next rounds. But their locked amount will be accounted into `total_stake`, which will dilute the original rewards per share.

Recommendation

Should prevent adding LM locked state with zero locked days, because according to the current locked staking options, it will be equivalent to the LM liquid stake.

Mitigation Review Log

Fixed in [PR-197](#).

4.14 Excessive LP Fee Distribution to Locked LP

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic

Description

The `get_locked_lp_stakers_fee` function calculates `share_per_token` by combining both unlocked LP and weighted locked LP(`lp_staking.current_staking_round.total_stake`), but the `lp_organic_fee` remaining in the LP pool would also be shared by `nb_locked_tokens`, which means locked LP shares both `lp_organic_fee` and `locked_lp_stakers_fee`.

```
326 let non_locked_staked_tokens = lp_token_mint.supply -  
    ↪ lp_staking.nb_locked_tokens;  
327  
328 let total_lp_holders_shares =  
329     non_locked_staked_tokens +  
    ↪ lp_staking.current_staking_round.total_stake;  
330 ...  
331 let share_per_token =  
332     (lp_organic_fee as u128 * Cortex::RATE_POWER) /  
    ↪ total_lp_holders_shares as u128;
```

[programs/adrena/src/state/cortex.rs#L326-L336](#)



Impact

There will be more rewards distributed as USDC extra rewards to the locked LP staking pool, but not enough going to the ALP accruing value.

Recommendation

Directly use `lp_token_mint.supply + lp_staking.current_staking_round.total_stake` as `total_lp_holders_shares`, instead of using `non_locked_staked_tokens`.

Mitigation Review Log

Adrena Team: the PR to fix the distribution [PR-208](#).

Offside Labs: Fixed.

4.15 Increase Position with Swap Checks Incorrect Custody Flag

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic

Description

The `open_or_increase_position_with_swap_long` and `open_or_increase_position_with_swap_short` instructions check for `receiving_custody.allow_trade == 1`. However, `receiving_custody` is only used for collateral swapping and not for position opening, the actual position opening uses `principal_custody`.

Impact

If `receiving_custody`'s `allow_swap` is 1 but `allow_trade` is not 1, positions that should be able to open and increase may be blocked from these operations through these two instructions.

Recommendation

It should check `principal_custody`'s `allow_trade` flag instead of `receiving_custody`'s flag.

Mitigation Review Log

Adrena Team: [PR-171](#).

Offside Labs: Fixed.



4.16 Should Use High Price to Calculate Collateral Withdrawal

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic

Description

The `remove_collateral_long` instruction uses low price to calculate the amount of collateral tokens for withdrawal, which results in more collateral being withdrawable per USD value.

[programs/adrena/src/instructions/public/position/remove_collateral_long.rs#L154-L157](https://github.com/Adrena/programs/adrena/src/instructions/public/position/remove_collateral_long.rs#L154-L157)

Impact

For example, if the oracle price of collateral remains unchanged but confidence increases, users can withdraw more collateral. Although this is capped by the initially deposited collateral, the increased `collateral_usd` means users can maintain the same position with fewer collateral tokens.

Recommendation

Use `token_price.high()` as `max_collateral_price` .

Mitigation Review Log

Adrena Team: [PR-172](#) .

Offside Labs: Fixed.

4.17 Short Positions Should Also Use High Collateral Price for Accounting Update

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The input parameter `collateral_token_price` is only used to check the `locked_amount_usd` is capped by `collateral_custody.pricing.max_position_locked_usd` . But the instructions `open_position_short` and `increase_position_short` all use `collateral_token_price_low` as the input. This results in a smaller `locked_amount_usd` , which could bypass the upper limit check.



- [programs/adrena/src/instructions/public/position/increase_position_short.rs#L327-L327](#)
- [programs/adrena/src/instructions/public/position/open_position_short.rs#L348-L348](#)

Recommendation

Use `collateral_token_price_high` when calling `custody.update_accounting_after_open_position_short` in the `open_position_short` and `increase_position_short`.

Mitigation Review Log

Adrena Team: [PR-173](#).

Offside Labs: Fixed.

4.18 Set SL Instruction Should Ensure `close_position_price` Parameter Is Not Zero

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

When `set_stop_loss_long / short` instruction builds `ClosePositionLong` or `ClosePositionShort` for the thread, it doesn't check if the input parameter `params.close_position_price` is zero, while the `close_position_long` and `close_position_short` instructions perform this check.

```
280     if params.price.is_some() && params.price.unwrap() == 0 {
281         return Err(ProgramError::InvalidArgument.into());
282     }
```

[programs/adrena/src/instructions/public/position/close_position_long.rs#L280-L282](#)

Impact

Due to zero price limit typically indicating no slippage check, this missing check will cause automated SL orders to fail execution, resulting in losses.

Recommendation

If `params.close_position_price` is not `None`, ensure its not zero.



Mitigation Review Log

Offside Labs: Fixed after removing the Sablier threads.

4.19 Remove Liquidity LP Fees Distribution is Inaccurate

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic

Description

Instruction `remove_liquidity` calls `calculate_fee_distribution` function after the CPI for burning LPs. This leads to inaccurate fee distribution due to changes in the total supply of `lp_token_mint`.

For `remove_liquidity`, due to not reloading `lp_token_mint` after the burn CPI, `calculate_fee_distribution` uses an overestimated `lp_token_mint.supply` value for fee distribution calculation.

This results in an underestimated `share_per_token` in the `get_locked_lp_stakers_fee` function, ultimately leading to an underestimated `locked_lp_stakers_fee` and an overestimated `lp_organic_fee`.

Impact

This issue in the `remove_liquidity` instruction causes LP stakers to receive less fees than expected.

Recommendation

In the `remove_liquidity` instruction, call `lp_token_mint.reload()` before calling `calculate_fee_distribution`.

Mitigation Review Log

Fixed in [PR-193](#).

4.20 Borrow Rate Not Updated After Fee Distribution

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic



Description

In the `distribute_fees` function, fees to be distributed are first deducted from `custody.assets.owned` before distribution.

```
380 custody.load_mut()?.assets.owned -= fees_to_redistribute_manually;
```

[programs/adrena/src/state/cortex.rs#L380-L380](#)

If `swap_required` is false, this custody's borrow rate will not be updated after `assets.owned` decreases.

Impact

Borrow rate will be lower than expected after the following operations, which will reduce the accumulation of borrowing interest:

1. [programs/adrena/src/instructions/public/liquidity/swap.rs#L459-L460](#)
2. [programs/adrena/src/instructions/public/liquidity/remove_liquidity.rs#L339-L339](#)
3. [programs/adrena/src/instructions/public/liquidity/add_liquidity.rs#L369-L369](#)
4. [programs/adrena/src/instructions/public/position/close_position_long.rs#L417-L417](#)
5. [programs/adrena/src/instructions/public/position/close_position_short.rs#L439-L439](#)
6. [programs/adrena/src/instructions/public/position/liquidate_long.rs#L378-L378](#)
7. [programs/adrena/src/instructions/public/position/liquidate_short.rs#L403-L403](#)

Recommendation

The `custody.update_borrow_rate` method at the above location can be moved to after `owned` reduction in the `distribute_fees` method.

Mitigation Review Log

Fixed in [PR-194](#).



4.21 External Fees Not Considered in AUM Check

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic

Description

The swap and add/remove liquidity instructions use `pool.check_token_ratio` method to verify if `custody.assets.owned` meets ratio restrictions after changes. However, since a portion of fees will be deducted from `custody.assets.owned` through `Cortex.distribute_fees` at the end of the instruction, the ratio check always under or over-estimates the owned changes.

1. In the swap instruction:

[programs/adrena/src/instructions/public/liquidity/swap.rs#L368-L380](#) `deposit_amount` should not include `fees_to_redistribute_manually` during ratio check, while `withdrawal_amount` should include it.

2. `remove_liquidity` instruction:

[programs/adrena/src/instructions/public/liquidity/remove_liquidity.rs#L291-L291](#) `withdrawal_amount` should include `fees_to_redistribute_manually` during ratio check.

3. `add_liquidity` instruction:

[programs/adrena/src/instructions/public/liquidity/add_liquidity.rs#L287-L287](#) `deposit_amount` should not include `fees_to_redistribute_manually` during ratio check.

Instructions `swap` and `remove_liquidity` check `assets.owned - assets.locked >= withdrawal_amount` when withdrawing tokens to avoid insufficient custody tokens to pay for profits. However, this check does not include `fees_to_redistribute_manually` in `withdrawal_amount`. When `fees_to_redistribute_manually` is deducted in the `distribute_fees` method, it may lead to `assets.owned < assets.locked`. This may result in insufficient custody collateral for probable profits:

1. [programs/adrena/src/instructions/public/liquidity/remove_liquidity.rs#L296-L296](#)
2. [programs/adrena/src/instructions/public/liquidity/swap.rs#L385-L385](#)

In the instructions `add_liquidity` and `remove_liquidity`, `pool.aum` will be updated before fee distribution, where this AUM has not deducted the `fees_to_redistribute_manually` that will be deducted by `distribute_fees`:

1. [programs/adrena/src/instructions/public/liquidity/remove_liquidity.rs#L347-L347](#)



2. [programs/adrena/src/instructions/public/liquidity/add_liquidity.rs#L377-L377](#)

The `get_assets_under_management_usd` method also doesn't account for the part of the exit fees distributed to external parties when calculating the latest AUM, while the `loss_usd` returned by `get_pnl_usd` includes these external fees.

Recommendation

Recommend including or excluding external fees (the local variable `fees_to_redistribute_manually` in `distribute_fees` method) in the above code locations involving AUM checks and updates.

Mitigation Review Log

Fixed in [PR-210](#).

4.22 Possible Updating on Unintended LockedStakes when Claiming Genesis Rewards

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic

Description

In `claim_stakes` IX, the `params.locked_stake_indexes` is used to select the `locked_stake` for genesis claim.

```
208 for locked_stake in user_staking
209     .locked_stakes
210     .iter_mut()
211     .filter(|stake| stake.amount != 0)
212     .enumerate()
213     .filter_map(|(index, stake)| {
```

[programs/adrena/src/instructions/public/staking_p/claim_stakes.rs#L208-L213](#)

While iterating through `user_staking.locked_stakes`, a round of filtering is first performed based on `stake.amount != 0`, and then the filtered intermediate result vector is enumerated. The `index` from the enumeration is used to query `params.locked_stake_indexes`.

However, the `index` used in the query corresponds to the intermediate filtered result vector rather than the original `user_staking.locked_stakes`. As a result, the `index` may be incorrect and could mismatch the actual intended `index` in `params.locked_stake_indexes`.



Impact

The incorrect `index` in the loop might prevent the intended `locked_stake` rewards from being claimed during this `claim_stakes` operation. Furthermore, it becomes impossible to claim the intended `locked_stake` again in this round since the `claim_time` has already been updated.

Recommendation

It is recommended to reorder the `enumerate` and `filter` to ensure the correct `index` is obtained in the final `filter_map`.

Mitigation Review Log

Fixed in [PR-197](#).

4.23 Liquidity Operations Moving Towards Target Ratio May be Rejected

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic

Description

In the `pool.get_fee` method, when the new token ratio falls outside the min/max ratio range, an error will be thrown and the entire instruction will be rejected.

[programs/adrena/src/state/pool.rs#L1101-L1123](#)

However, the current liquidity operation may already be optimizing the token ratio towards the target ratio.

Impact

When pool assets are severely imbalanced, it's not possible to add/remove/swap relatively small amounts of liquidity, even if such liquidity operations would be beneficial for restoring the target ratio.

Recommendation

Should use the check logic similar to the `check_token_ratio` function:

```
519     if new_ratio < self.ratios[token_id].min {
520         Ok(new_ratio >= self.get_current_ratio(custody, token_price?))
521     } else if new_ratio > self.ratios[token_id].max {
522         Ok(new_ratio <= self.get_current_ratio(custody, token_price?))
```



[programs/adrena/src/state/pool.rs#L519-L522](#)

But this modification of the ratio check will lead to the case where `new_ratio > max_ratio` or `new_ratio < min_ratio` and then `fee_adjustment` will be greater than `fee_max`, or less than `fee_min - fee_max`. Since negative `fee_adjustment` values are not currently effective, we don't need to consider the second case of smaller negative values.

So it needs to cap the `final_fee` by the `fee_max`.

Mitigation Review Log

Fixed in [PR-212](#).

4.24 Unable to Guarantee Operations Sequence About Staking

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Logic

Description

The onchain program doesn't guarantee the operations sequence of rounds resolving, locked stakes finalization and removal.

We list the possible impacts and affected cases in the following impact section.

The current RPC server *MrSablierStaking* can handle sequencing issues well when there is no network congestion. Among the following affected scenarios, the first situation would still be triggered under normal conditions due to race conditions.

Impact

1. If a locked stake will end after the current round can be resolved(after `current_staking_round.start_time + ROUND_MIN_DURATION_SECONDS`), and it is finalized before the current round is resolved. This will prevent users from claiming rewards from rounds they should be eligible for, because when a round is resolved, its end time is set to the current time, which will be greater than the locked stake's end time.
2. If a locked stake will end before the current round ends, and it isn't finalized timely before the current round is resolved, the stake amount of this locked stake will dilute the reward rate of the current round, and left a part of rewards until the end of 32 rounds. And if this locked stake is still not finalized in the next round, when its finalized, the stake amount will not be deducted from the `total_stake` of `current_staking_round`. Because it doesn't qualify for rewards from the current round.
3. During round resolution, it iterates through Staking `resolved_staking_rounds` to



find the first empty index, therefore the rounds in `resolved_staking_rounds` are not strictly ordered by time. If a user adds liquid stake in round 1 but doesn't claim rewards after round 1 resolution, and there happens to be an empty slot in the `resolved_staking_rounds` array before round 1 when round 2 is resolved, then round 2 will be positioned before round 1.

After round 2 is resolved, when user claims stakes and iterates through `resolved_staking_rounds`, round 2 will be processed first, resetting `user_staking.liquid_stake.overlap_amount` to 0. When processing round 1, the `stake_amount` won't subtract the original `overlap_amount` value. Moreover, since `overlap_amount` is not included in round1's `total_stake`, the rewards per stake share for round1 will be larger, resulting in users receiving more rewards than intended.

Recommendation

For the first scenario above, even with the RPC server running normally, it can still occur due to race conditions. To prevent this situation, it's recommended to check if `staking.current_staking_round_is_resolvable(current_time)` has been reached during locked stake finalization. At this point, locked stakes should be prevented from being finalized if they qualify for rewards from round and the end time $\geq$ `current_staking_round.start_time + ROUND_MIN_DURATION_SECONDS` (this condition is not included in the `qualifies_for_rewards_from` method).

For the second and third cases, they only occur when there is network congestion or when the RPC server remains offline.

Mitigation Review Log

Adrena Team: Acknowledged.

4.25 Informational and Undetermined Issues

No Checks on Opening Long Positions With Stable Custodies

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

The `open_position_short` instruction prohibits opening positions with stable custody, but the `open_position_long` instruction lacks the same check. Due to separate branch logic for stable custody in the `Pool.get_assets_under_management_usd` function, the PnL from these long positions with stable custodies is not included in the AUM calculation.

Mitigation Review Log: Fixed in [PR-175](#).



Suggest Checking Both Positions Open Time and Update Time

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

In `close_position` related instructions, `MIN_POSITION_OPEN_TIME_SECONDS` check is only performed against `position.open_time`. Users can prepare a dust position (\$10 size) in advance, and when real arbitrage or attack opportunities arise, bypass this check by instantly closing the position after using increase position instructions. So suggest checking both open time and non-zero update time of positions.

`close_position_short` Not Update `collateral_custody.collected_fees.borrow_usd`

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

Following instruction `liquidate_short`, the following state update should be added in the `close_position_short` instruction, [programs/adrena/src/instructions/public/position/close_position_short.rs#L397-L397](https://github.com/Adrena/programs/blob/master/src/instructions/public/position/close_position_short.rs#L397-L397)

```
collateral_custody.collected_fees.borrow_usd +=  
    exit_numbers.borrow_fee_usd;
```

`OpenOrIncreasePositionWithSwapShort` Instruction Missing `protocol_fee_recipient` Check

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

The `OpenOrIncreasePositionWithSwapShort` instruction doesn't check if the account `protocol_fee_recipient` is equal to `cortex.protocol_fee_recipient`.

Although incorrect `protocol_fee_recipient` will ultimately fail in the following CPIs, it's recommended to add `has_one = protocol_fee_recipient` constraint in the cortex account.

Suggested Add Price Validation for SL Parameters

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

For `set_stop_loss_long` instruction, the parameter `SetStopLossLongParams.stop_loss_limit_price` should always be not less than the parameter `SetStopLossLongParams.close_position_price` if its not None.

For `set_stop_loss_short` instruction, the parameter `SetStopLossShortParams.stop_loss_limit_price` should always be not great than the parameter `SetStopLossShortParams.close_position_price` if its not None.



Otherwise, SL orders with such settings will always fail during execution.

Incorrect Validation in `remove_pool`

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

In the `remove_pool` instruction, the following validation logic is implemented:

```
53     require!(  
54         ctx.accounts.pool.load()?.custodies.is_empty(),  
55         AdrenaError::InvalidPoolState  
56     );
```

[programs/adrena/src/instructions/admin/pool/remove_pool.rs#L53-L56](#)

However, the validation will always fail because `custodies` is defined as a constant-length array, and `is_empty()` will always return false.

Incorrect Validation in `remove_custody`

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

In the `remove_custody` instruction, the following validation logic is performed:

```
90     if ctx.accounts.pool.load()?.ratios.is_empty()  
91         || params.ratios.len() != ctx.accounts.pool.load()?.ratios.len()  
92     - 1  
93     {  
94         return Err(ProgramError::InvalidArgument.into());  
95     }
```

[programs/adrena/src/instructions/admin/custody/remove_custody.rs#L90-L94](#)

However, the validation will always pass because `ratios` is defined as a constant-length array, and `is_empty()` will always return false. Additionally, the following comparison between `len()` will always be true since both `ratios` are constant-length arrays with equal length.

Incorrect Unique Custody Check in `Pool::validate`

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

In the `Pool::validate()` method, the following logic is implemented to ensure that custodies are unique:



```
194 // check custodies are unique
195 for i in 1..self.custodies.len() {
196     if self.custodies[i] != Pubkey::default()
197       && self.custodies[i..].contains(&self.custodies[i - 1])
198     {
199         return false;
200     }
201 }
```

[programs/adrena/src/state/pool.rs#L194-L201](#)

However, there is a flaw in this implementation. The validation can be bypassed by the following operation sequence:

```
add_c1, add_c2, add_c3, remove_c1, remove_c2, add_c3
```

It can be fixed by replacing `self.custodies[i]` with `self.custodies[i - 1]` in line 196.

While there is no impact currently because the unique property is also enforced by additional validation logic elsewhere in the contract:

```
113 if pool.get_token_id(&ctx.accounts.custody.key()).is_ok() {
114     return Err(ProgramError::AccountAlreadyInitialized.into());
115 }
```

[programs/adrena/src/instructions/admin/custody/add_custody.rs#L113-L115](#)

Recommend to Check if the Locked State Has Ended When Finalizing With `early_exit`

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic

If the locked state has ended but the owner calls the `finalize_locked_stake` instruction with `params.early_exit=true`, it can reset the `locked_stake.end_time` to current time. However, this can not be exploited now, because in the `calculate_early_exit_fee_amount` function a negative `time_remaining` will cause the following calculation of `fee_rate` to overflow and fail.

OraclePrice::normalize Doesn't Update the Confidence

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

[programs/adrena/src/state/oracle.rs#L160-L160](#)

The `OraclePrice::normalize` function adjusts the price exponent but not the confidence exponent, which results in amplified confidence.

Mitigation Review Log:



Adrena Team: We would like to remove this normalize, as we would rather just overflow in case the price returned from the feed is “incorrect” (too big).

Instruction `add_vest` Can Add Vests Which Can be Almost Fully Unlocked Immediately

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic

The `add_vest` instruction doesn't check the numerical relationship between the `unlock_start_timestamp` parameter and the `current_time`.

If admin sets `params.unlock_start_timestamp` to 0, this vest can be almost immediately unlocked in full.

Insufficient Precision of PositionsAccounting.total_quantity

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Precision

Functions `Pool.update_accounting_after_open_position_short/long` use the following codes to update the opening price and position size (equivalent custody tokens) across all positions:

```
508 let position_size_usd = math::scale_to_exponent(  
509     position.size_usd,  
510     -(Cortex::USD_DECIMALS as i32),  
511     -(Cortex::PRICE_DECIMALS as i32),  
512 )?;  
513  
514 let quantity = (position_size_usd as u128 * Cortex::BPS_POWER) /  
515     position.price as u128;  
516 accounting.weighted_price += position.price as u128 * quantity;  
517  
518 accounting.total_quantity += quantity;
```

[programs/adrena/src/state/custody.rs#L508-L518](#)

Here, `quantity` is calculated as a scalar multiplied by `Cortex::BPS_POWER`, which means that although `PositionsAccounting.total_quantity` is a `u128` type, its decimals is only 4.

This means that for each position, the portion of size below 0.0001 custody tokens won't have its PnL reflected in the AUM. For example, at current prices 0.0001 BTC is worth about \$7, so with 1000 positions with the BTC custody, approximately \$7000 of OI's PnL from these fractional amounts would not be reflected in the AUM. This could potentially cause losses for LPs.

Mitigation Review Log:



Adrena Team: This would require an update in the decimals + a patch to the value. This seems to only affect bitcoin, and our min position value is 9.5\$ This is fixable if we can bundle a program upgrade and the execution of the value patching in a single proposal in the governance.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

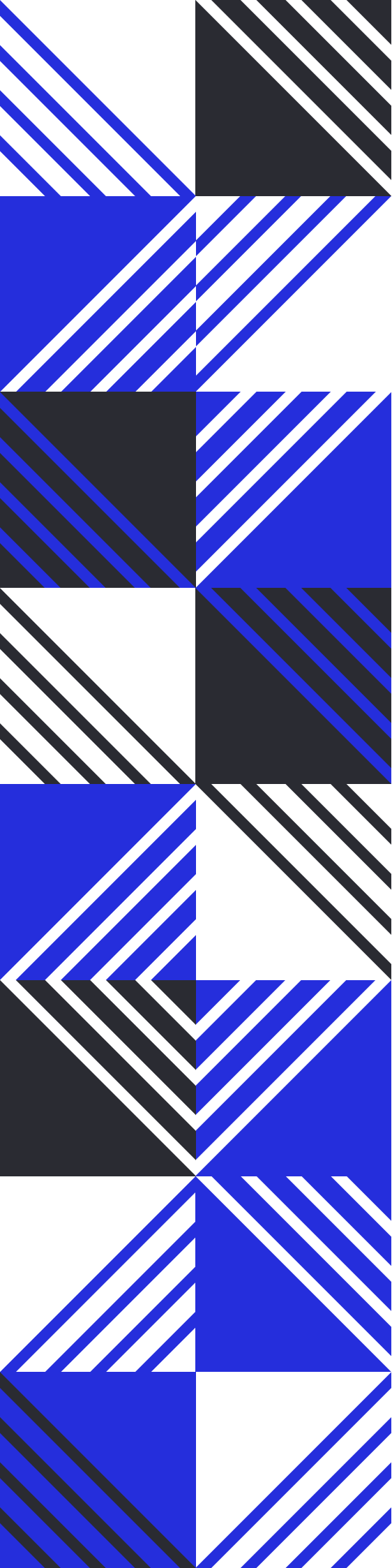
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs